CHAPTER 12

I/O: Interrupt & DMA

I/O interface · memory-mapped vs isolated · programmed I/O · interrupt-driven · DMA (burst, cycle stealing) · CPU overhead calculations

QUICK TRICK

Three ways: **programmed I/O** (CPU polls, wastes time), **interrupt-driven** (device interrupts the CPU per transfer), **DMA** (controller moves a whole block, one interrupt at the end). Overhead = **(events per second × cycles per event) ÷ clock rate**.

12.1 I/O interface

● **LEVEL 1 · BEGINNER**

An **I/O controller (interface)** sits between the CPU and a device. It has **status**, **command/control** and **data** registers. **Analogy.** A receptionist who takes messages for a busy manager.

	Memory-mapped I/O	Isolated (port-mapped) I/O
Address space	Shared with memory	Separate I/O space
Instructions	Ordinary LOAD / STORE	Special IN / OUT
Control signal	Same memory read/write lines	Separate I/O read/write line
Drawback	Uses up some memory addresses	Needs extra instructions and a control line

12.2 Programmed I/O and interrupts

● **LEVEL 2 · EXAM**

- **Programmed I/O (polling):** the CPU loops reading the status register until the device is ready. Simple, but the CPU is **busy-waiting**.
- **Interrupt-driven I/O:** the CPU does other work; the device raises an **interrupt request (IRQ)** when ready. Efficient for slow, sporadic devices.

```
Interrupt sequence
1. Device raises IRQ              2. CPU finishes the current instruction
3. CPU acknowledges, identifies the source (vector number)
4. CPU saves PC and flags (state)  5. Jumps to the Interrupt Service Routine (ISR)
6. ISR runs; returns with IRET (restores PC and flags)
```

- **Vectored** interrupt: the device supplies the ISR address (vector). **Non-vectored:** a fixed ISR polls devices to find the source.
- **Priority: daisy chain** (the device nearest the CPU wins), or a priority encoder. **Maskable** interrupts can be disabled; **non-maskable (NMI)** cannot. Nested interrupts allow a higher priority to pre-empt an ISR.
- **Exceptions/traps** (divide by zero, system call) are internal (software) interrupts. Number of vectors with an 8-bit vector number = 2^8 = **256**.

12.3 DMA

● **LEVEL 2 · EXAM**

The problem. Moving a large block word by word through the CPU wastes it. **The idea.** A **DMA controller** moves data between the device and memory directly. It has an **address register**, a **word count register** and a **control register**.

```
1. CPU programs the DMA controller: memory address, count, direction  (then goes on working)
2. Device is ready -> DMA asks for the bus (bus request / HOLD)
3. CPU grants it (bus grant / HLDA)        4. DMA transfers words directly to/from memory
5. After the last word DMA raises ONE interrupt: transfer complete
```

DMA mode	How the bus is used	CPU effect
Burst (block)	DMA holds the bus for the **whole block**	CPU locked out for the entire block
Cycle stealing	DMA takes the bus for **one memory cycle** per word	CPU slowed slightly each time
Transparent	DMA uses only cycles when the CPU is not using the bus	No slowdown, complex hardware

12.4 Overhead calculations

● LEVEL 3 · GATE

Polling a mouse: 30 polls/s × 400 cycles on a 500 MHz CPU = $12{,}000/5\times10^8$ = **0.0024%** of the CPU.

Polling a disk delivering 4 MB/s in 16 B chunks: $4\times10^6/16$ = 250,000 polls/s × 400 cycles = 10^8 cycles/s = **20%** of 500 MHz.

DMA for the same disk, 8 KB transfers, setup 1000 cycles + completion interrupt 500 cycles: transfers/s = $4\times10^6/8192 \approx 488$; overhead = 488 × 1500 = 732,000 cycles/s ≈ **0.146%**.

Cycle stealing: device 800 KB/s, 4-byte words → 200,000 words/s; each steals a 100 ns memory cycle: 200,000 × 100 ns = 20 ms per second = **2%** of the CPU time. **Burst:** a block of 1000 words with a 100 ns cycle blocks the CPU for **100 µs**.

12.5 Common mistakes

COMMON MISTAKE

- Saying DMA needs one interrupt per word (it needs **one** per block).
- Forgetting that the CPU still sets up the DMA controller.
- Mixing bytes per second and words per second in the overhead formula.
- Treating cycle stealing and burst mode as the same.

QUICK TRICK

Memory line: "Polling asks, interrupt tells, DMA does the carrying."

GATE QUICK CHECK — Chapter 12

```
overhead = (events/s x cycles/event) / f_clk
polling: busy-wait | interrupt: per transfer | DMA: setup + 1 interrupt per block
burst: CPU blocked for whole block | cycle stealing: 1 memory cycle per word
memory-mapped: LOAD/STORE | isolated: IN/OUT | daisy chain: nearest device wins
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which I/O method wastes CPU time busy-waiting?

Q2. How does the DMA controller tell the CPU a transfer has finished?

Q3. In a daisy-chain priority scheme which device has the highest priority?

Q4. How many interrupt vectors can an 8-bit vector number address?

● LEVEL 2 · EXAM

Q5. A mouse is polled 30 times per second at 400 cycles per poll on a 500 MHz CPU. CPU fraction used?

Q6. A disk delivers 4 MB/s in 16 B chunks and is polled for each chunk (400 cycles per poll, 500 MHz). CPU fraction?

Q7. Cycle-stealing DMA: 800 KB/s device, 4-byte words, 100 ns memory cycle. CPU time lost?

Q8. Burst DMA of 1000 words, 100 ns per word. How long is the CPU locked out?

● LEVEL 3 · GATE

Q9. DMA for the 4 MB/s disk: 8 KB transfers, 1000 cycles setup + 500 cycles completion interrupt, 500 MHz CPU. CPU fraction?

Q10. Why can memory-mapped I/O use ordinary LOAD/STORE instructions?

Q11. List the order of events when an interrupt is taken.

Answers

Q	Solution
1	**Programmed I/O (polling)**.
2	With an **interrupt**.
3	The one **closest to the CPU**.
4	**256**.
5	$30 \times 400 / 5 \times 10^8$ = **0.0024%**.
6	$250{,}000 \times 400 = 10^8$ cycles/s → **20%**.
7	200,000 × 100 ns = 20 ms/s = **2%**.
8	**100 µs**.
9	Transfers/s = $4 \times 10^6 / 8192 \approx 488$. Cycles = 488 × 1500 ≈ 732,000/s → **0.146%**.
10	Device registers occupy addresses in the same address space as memory, so ordinary memory instructions reach them.
11	Finish current instruction → acknowledge and get vector → save PC and flags → run the ISR → restore state and return.

Next: Chapter 13 — Master Formula Sheet and Mixed Practice.

AMJESH singhamjesh.github.io